GENERATIVE ARTIFICIAL INTELLIGENCE

PROOF OF CONCEPT (PoC) — CONTINUOUS ASSESSMENT-3

Semester VII — Information Technology | Continuous Assessment | Maximum Marks: 10

1. Student Details

Particular	Details
Student Name(s)	Chaitanya Kelkar; Jay Kerkar; Yash Kolekar
Roll No(s)	15; 16; 17 (respectively)
Division	D20A
Batch	A
Faculty Name	Prof. Dipti Karani
Date of Submission	2 November 2026
PoC Title	Crucible: An Agentic, Self-Healing Generative-AI Development Environment for Web3 dApps
Selected Self-Learning Topic	☒ Prompt Engineering ☒ RAG Basics ☒ AI Hallucinations & LLM Limitations ☒ Responsible AI ☒ Other: Agentic AI (tool-calling LLM agents over MCP)
Individual / Group	Group (3 students)

2. Problem Identification

2.1 Problem Statement

Building a decentralised application (dApp) today forces a developer to stitch together five disconnected tools: a code editor for Solidity and frontend code, a terminal running a local blockchain node, a browser to test the frontend, a wallet extension (with a manually configured RPC endpoint, imported keys and nonce resets), and a block explorer to debug failed transactions. These tools do not communicate with each other, and an ordinary AI coding assistant sees only text files: it has no awareness of chain state, wallet balances or deployment history, and it cannot observe a transaction revert, let alone repair it. Developers therefore spend more time wiring tools together than writing code, and Web3 development remains slow for experts and inaccessible to newcomers.[1]

2.2 Target Users

Smart-contract and dApp developers, from beginners learning Solidity to experienced Web3 engineers who want faster build–test–debug cycles; hackathon and student teams prototyping on testnets.

2.3 Why is this Problem Important?

Smart-contract bugs are expensive and difficult to diagnose because contracts are immutable once deployed and failures surface as opaque transaction reverts. Shortening the loop *write* $\rightarrow$ *compile* $\rightarrow$ *deploy* $\rightarrow$ *interact* $\rightarrow$ *debug* lowers the entry barrier to Web3, and an assistant that can observe execution results (instead of only emitting text) can catch and fix defects automatically before a developer wastes time on them. It also makes a

concrete case study for tool-using generative-AI agents, where hallucinated output must be checked against ground truth produced by a compiler and an EVM.

3. Proposed GenAI Solution

3.1 Proposed Solution

Crucible is a browser-based agentic development environment: the user describes a dApp in natural language and a large language model (LLM) agent builds it step by step. The backend assembles a system prompt (role, tool documentation, workflows and a live index of workspace files) plus the chat history, and streams the request to an LLM through the Vercel AI SDK. The model acts only by calling tools exposed over the Model Context Protocol (MCP): writing files, compiling Solidity, starting a per-workspace Hardhat chain, deploying, calling contracts and recalling stored fixes. When a transaction reverts, a controller forces the model through a fixed seven-step repair sequence (snapshot, trace, recall, patch, compile, revert, redeploy) with at most three attempts. Every event is streamed to the browser so that all actions stay visible and overridable.

Item	**Details**
Primary GenAI Concept Used	Tool-using LLM agent [2, 3] (function calling over MCP [4]) steered by prompt engineering: a long role-and-workflow system prompt, plus per-step tool restriction (`activeTools` / forced `toolChoice`) that turns the model into a constrained self-repair state machine.
Why was this concept selected?	Generating Solidity from text alone is unreliable. Grounding the LLM in real tool results (compiler errors, EVM traces, revert reasons) lets the system detect hallucinated or buggy code automatically and correct it, instead of trusting the first answer. MCP gives a uniform, schema-validated way to expose chain, compiler, wallet and memory services to the model.
Expected Input / Output	**Input:** a natural-language request (e.g. "Build me a token vault with deposit, withdraw and a 24-hour withdrawal cooldown") together with the current workspace files and chat history. **Output:** Solidity and frontend source files written to the workspace, compiled artifacts, deployed contract addresses, a live dApp preview, and a streamed trace of every tool call, revert and patch.

Syllabus mapping. The PoC covers: Module 1 (NLP applications, transformer/GPT overview, pre-trained models used through prompting rather than fine-tuning, GPT-based dialogue system); Module 2 (text/code generation, transfer learning, privacy and security); Module 5 (prompt engineering, RAG-style retrieval of past fixes from memory, hallucinations and LLM limitations, conversational AI); Module 6 (developer-tooling case study, zero/few-shot use, agentic AI trends, Responsible AI). Modules 3 (autoencoders, VAEs) and 4 (GANs) are not covered.

4. System Architecture / Workflow

Figure 1 follows the suggested flow of the template (User/Input $\rightarrow$ Preprocessing $\rightarrow$ GenAI Model $\rightarrow$ Processing $\rightarrow$ Output $\rightarrow$ Evaluation). Steps are numbered in execution order.

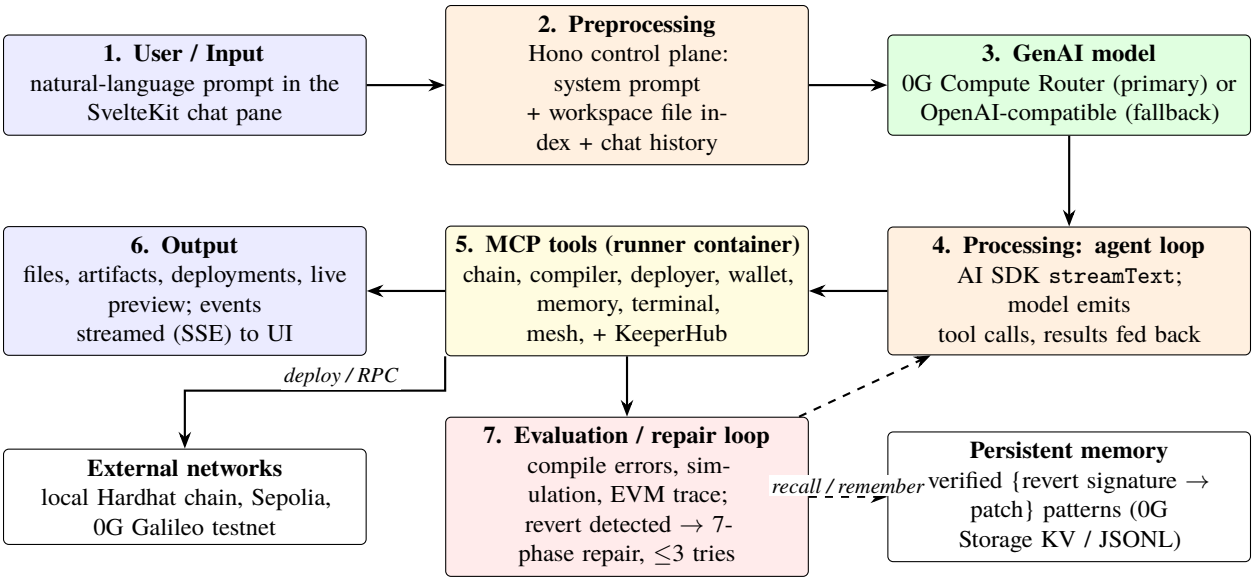


Figure 1: System architecture and workflow of the Crucible PoC. Solid arrows show the main request flow; dashed arrows show the repair feedback and memory lookups.

Figure 2 details the signature mechanism of the PoC. A revert returned by `deploy_local`, `call_contract` or `send_tx_local` starts a controller that, at every model step, exposes *only one* tool and forces the model to call it, so the model cannot skip or reorder the repair phases. A successful verification emits `patch_verified` and allows one constrained "resume" step so the original task continues; a fourth failed attempt emits `repair_failed` and ends the turn.

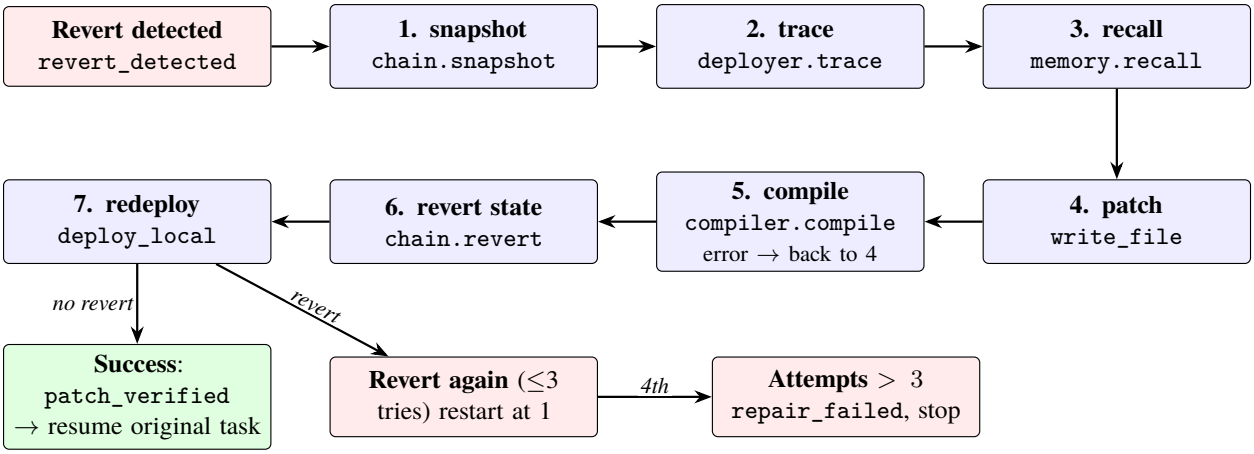


Figure 2: Forced seven-phase self-healing repair loop implemented in `packages/agent/src/loop.ts`.

5. Tools, Technologies and Model

Versions are the semantic-version ranges declared in the repository's `package.json` files.

Component	Technology Used
Programming Language	TypeScript (`typescript` ^5.7.3) executed directly by Bun 1.3.6; Solidity 0.8.x [5] for the generated contracts; JavaScript/Svelte for the generated frontend
GenAI Model	Transformer-based [6] chat-completion LLM selected at run time through environment variables: primary model from the 0G Compute Router (`OG_MODEL`; the project documentation cites `qwen3.6-plus` and `GLM-5-FP8` as examples), or any model on an OpenAI-compatible endpoint (`OPENAI_MODEL`) as a degraded-mode fallback. No model is trained or fine-tuned in this project.
Framework / Library	Vercel AI SDK [7] (`ai` ^6.0.168, `@ai-sdk/openai` ^3, `@ai-sdk/mcp` ^1.0.36); MCP server SDK (`@modelcontextprotocol/server` 2.0.0-alpha.2); Hono ^4.12; Zod ^4; viem ^2.48; Hardhat [8] ^3; Prisma ^7.8; better-auth ^1.6 (Sign-In with Ethereum); dockerode; Turborepo ^2.5
Dataset / Documents	No training dataset. Inputs at run time are the user's prompts, the workspace files, and the persisted chat history. A store of verified "revert signature → patch" patterns acts as the agent's memory (0G Storage KV when configured, otherwise a local JSONL file); `scripts/seed-memories.sh` seeds demo patterns.
API / Platform	0G Compute Router [9] (OpenAI-compatible REST); KeeperHub hosted MCP (post-deploy automation); Docker (one runner container per workspace); PostgreSQL; local Hardhat chain, Sepolia and 0G Galileo testnets
User Interface	SvelteKit ^2.57 / Svelte ^5.55 web app with Tailwind CSS ^4.2, chat rail, CodeMirror 6 editor, live preview iframe, transaction inspector and an xterm.js ^6 terminal
Other Tools	Bun test runner, ESLint, Prettier, GitHub Actions CI, Vite ^8 (per-workspace preview server), Portless local URLs

6. Implementation

6.1 Implementation Steps

1. **Workspace provisioning.** The backend (`packages/backend`, Hono + Prisma) creates a real directory `contracts/`, `frontend/`, `.crucible/` per workspace, starts one Docker runner container hosting the MCP servers, and a Vite preview server. Access is gated by Sign-In with Ethereum sessions [10].
2. **Prompt assembly.** On `POST /api/prompt`, `buildSystemPrompt()` creates the system prompt (role, tool catalogue, local/0G/Sepolia/KeeperHub workflows, repair rules, mesh rules) and appends an index of current workspace files with SHA-256 prefixes; the persisted chat history is replayed as prior messages.
3. **Streaming inference with tool calling.** `runAgentTurn()` opens an MCP client per runner service and

calls `streamText` (up to 40 steps) against the selected provider. A fetch wrapper normalises provider quirks (empty tool-call types, `reasoning_content`) and classifies 0G Router failures (rate limit, exhausted balance, outage) so the UI can show an honest fallback state.

4. **Tool execution.** The model writes Solidity/frontend files, compiles (`compiler.compile`), deploys (`deploy_local`,`deploy_sepolia`, `deploy_og_chain`), interacts (`wallet.call_contract`) and advances chain time (`chain.mine`); every tool call and result is emitted as a typed event over SSE.

5. **Self-healing and memory.** Reverts are detected by parsing tool results; `prepareStep` then restricts the model to the single tool of the current repair phase, a verified patch is stored through `memory.remember`, and one resume step returns control to the original task. A cleanup `finally` block reverts the chain snapshot if a turn is cancelled mid-repair.

6.2 Important Code / Algorithm

Listing 1 shows how the repair state machine constrains the model (abridged from `packages/agent/src/loop.ts`; type casts and comments omitted). Listing 2 shows how a revert is distinguished from an infrastructure error, and Algorithm 3 gives the controller in pseudocode.

```
const REPAIR_PHASE_TOOLS = {
  snapshot: { activeTools: ['snapshot'],     toolName: 'snapshot' },
  trace:    { activeTools: ['trace'],        toolName: 'trace' },
  recall:   { activeTools: ['recall'],       toolName: 'recall' },
  patch:    { activeTools: ['write_file'],   toolName: 'write_file' },
  compile:  { activeTools: ['compile'],      toolName: 'compile' },
  revert:   { activeTools: ['revert'],       toolName: 'revert' },
  deploy:   { activeTools: ['deploy_local'], toolName: 'deploy_local' },
};
...
streamText({
  model: openai.chat(config.model),
  system: buildSystemPrompt(files),
  stopWhen: [stepCountIs(40), () => repairFailed],
  prepareStep: ({ experimental_context }) => {
    ...
    const phaseConfig = REPAIR_PHASE_TOOLS[repairCtx.phase];
    return {
      activeTools: phaseConfig.activeTools,
      toolChoice: { type: 'tool', toolName: phaseConfig.toolName },
      experimental_context: repairCtx,
    };
  },
  ...
});
```

Listing 1: Per-phase tool restriction and forced tool choice (abridged).

```
export function extractDeployRevert(toolName: string, toolResult: unknown) {
  if (toolName !== 'deploy_local') return { reverted: false };
  const raw = toolResult as { isError?: boolean;
    content?: Array<{ type: string; text?: string }> };
  if (!raw.isError) return { reverted: false };
  const message = raw.content?.filter((c) => c.type === 'text')
    .map((c) => c.text ?? '').join('\n') ?? '';
  // Infrastructure errors ("node not running", "not found in artifact
  // store") must not trigger the repair loop.
  if (!/revert/i.test(message)) return { reverted: false };
  const txHash = /0x[0-9a-f]{64}/i.exec(message)?.[0] ?? '0x' + '0'.repeat(64);
  const reason = /reverted with reason:\s*"([^"]+)"/i.exec(message)?.[1];
  return { reverted: true, txHash, revertSignature: reason ?? message.slice(0, 200) };
}
```

Listing 2: Revert detection: only genuine EVM reverts start the repair loop (abridged).

```
on tool result r of deploy_local / call_contract / send_tx_local:
    if r is a genuine EVM revert:
        if no repair active:  start repair (attempts = 1); emit revert_detected
        else:                 attempts += 1
        if attempts > 3:      emit repair_failed; stop turn
        else:                 phase = snapshot
for each model step while phase != idle:
    restrict tools to TOOL[phase] and force the call
    snapshot -> trace -> recall -> patch(write_file) -> compile
        (compile error -> phase = patch) -> chain.revert -> redeploy
    on redeploy success: emit patch_verified; phase = resume (one step)
    on redeploy revert:  attempts += 1 (see above)
```

Listing 3: Repair-loop controller (pseudocode of `runAgentTurn`).

7. Working Demonstration

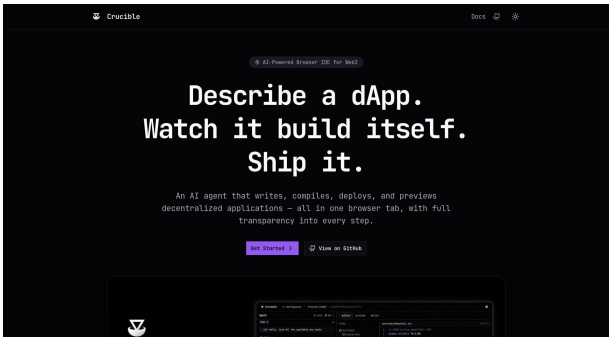


Figure 3: Crucible landing page.

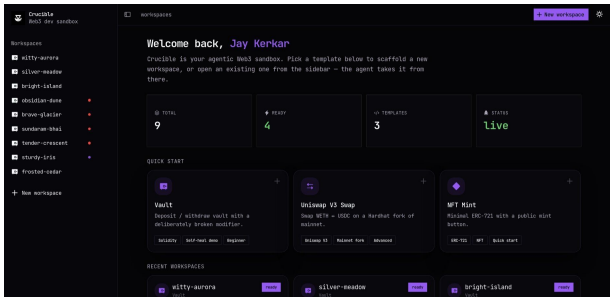


Figure 4: Workspace dashboard with the three starter templates.

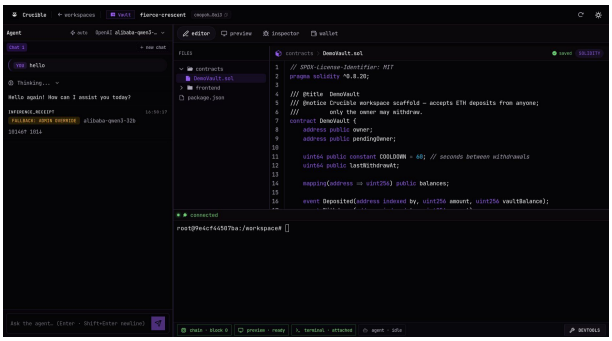


Figure 5: Screenshot 1 — Input. Agent workspace: prompt in the chat rail, generated `DemoVault.sol` in the editor, workspace terminal.

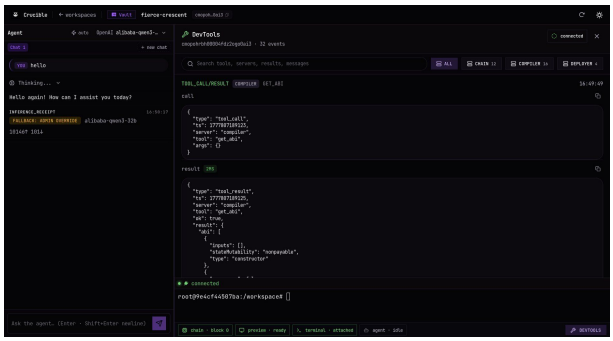


Figure 6: Screenshot 2 — Processing / Model. DevTools inspector: `tool_call`/`tool_result` pair and inference receipt.

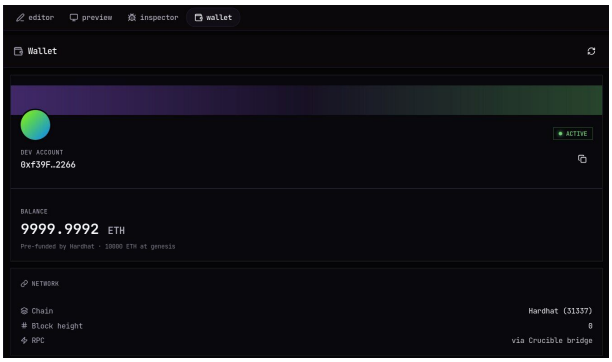


Figure 7: Screenshot 3 — Output. Embedded dev wallet on the workspace's local Hardhat chain.

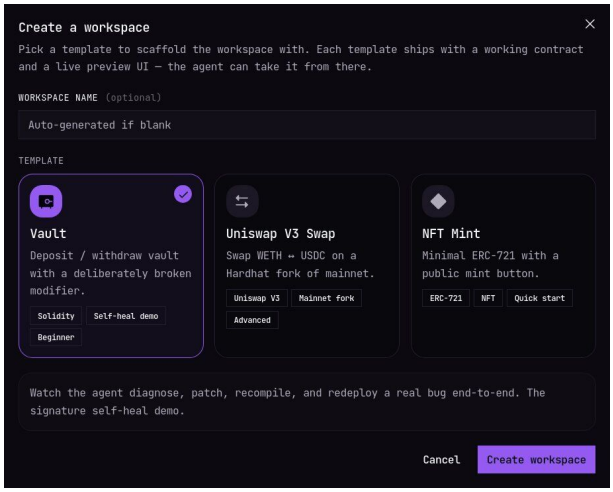


Figure 8: Create-workspace dialog with the Vault self-heal template.

How to run the PoC (from the repository's `README.md` and `packages/backend/.env.example`):

1. Clone the repository and copy the backend environment template to `.env`.
2. Set the inference credentials: `OG_API_KEY` and `OG_MODEL` (0G Compute Router), or `OPENAI_BASE_URL`, `OPENAI_API_KEY` and `OPENAI_MODEL` (fallback); also set `DATABASE_URL` and `BETTER_AUTH_SECRET`.
3. Build the runtime image: `bun run --cwd packages/backend runtime:build`.
4. Run `bun install`, then `bun run dev`, and open the app in the browser.

Demo / GitHub / Google Colab / Drive / Video Link:
Source repository: `https://github.com/WhyAsh5114/Crucible`
Demo video (YouTube): `https://www.youtube.com/watch?v=XDz7qSXFfp8`

8. Results and Evaluation

8.1 Test Cases

The repository's automated unit tests for the agent package (`packages/agent/test`) were run with `bun test`. Five representative cases are listed; the whole run reported 40 passed, 0 failed (69 assertions, 3 test files). See Section 8.3 for how the run was performed.

Test Case	Input	Expected Output	Actual Output	Result
1	`extractDeployRevert` on a `deploy_local` error "Transaction reverted: 0xaaa… reverted with reason: "Ownable: caller is not the owner""	`reverted: true`, tx hash extracted, signature = `Ownable: caller is not the owner`	Matched expected	Pass
2	`extractDeployRevert` on error text "node not running" (infrastructure error)	`reverted: false` (repair loop must not start)	Matched expected	Pass
3	`extractSnapshotId` on tool text `{"snapshotId": "snap_42"}`	`"snap_42"`	Matched expected	Pass
4	`extractSendTxRevert` on a `send_tx_local` result with `status: "reverted"`	`reverted: true`, signature contains "transaction reverted"	Matched expected	Pass
5	`classifyRouterError` with HTTP 429, 402 and 500 from the 0G Router	`rate_limited`, `balance_exhausted`, `provider_unavailable`	Matched expected	Pass

8.2 Evaluation Criteria

☒ Accuracy (rule-based revert detection and error classification) ☒ Response Time (tool-pipeline latency, Table 1) ☒ Other: functional correctness of the repair-loop logic (automated unit tests).

8.3 Results and Observations

The 40 unit tests of the agent package pass. They cover revert detection (including the rule that infrastructure errors never trigger a repair), snapshot and deployment metadata extraction, 0G Router error classification and trace filtering, and the content of the generated system prompt. The run was performed on 2 November 2026 using Bun 1.4.2 on a scratch copy of `packages/agent` and `packages/types` with dependencies resolved afresh, because `bun install --frozen-lockfile` on the repository itself reported that `bun.lock` was out of date. The backend, frontend and other MCP packages also contain tests, but they were not run for this report (the backend tests need a PostgreSQL instance).

The tests exercise the helper functions only: the repository itself notes that integration tests of the full revert-triggers-loop and three-failure-cap behaviour require mocking `streamText` and are out of scope of the current tests. The repository contains no systematic end-to-end benchmark of the generative component (e.g. fraction of prompts producing a compiling, deployable contract, repair success rate or hallucination rate), so none is claimed here.

Observations from the running instance (values read from the screenshots in Section 7): the workspace dashboard lists 9 workspaces, 4 of them ready, with 3 starter templates; the agent replied to the test prompt through the OpenAI-compatible fallback model `alibaba-qwen3-32b`, whose inference receipt reports 10,146 input and 101 output tokens (the large input figure reflects the long system prompt and tool catalogue); the DevTools inspector logged 32 events for that workspace (12 chain, 16 compiler, 4 deployer), and a compiler `get_abi` tool call returned in 2 ms; the embedded wallet shows a pre-funded Hardhat account on chain 31337. Scope of the evaluation: *accuracy* refers only to the deterministic helpers (all revert-detection, snapshot-

parsing and error-classification test cases matched their expected outputs, 40 of 40 tests passing); *response time* refers only to tool-call latency (the 2 ms `get_abi` call above and Table 1), not to LLM generation time. Together with the 40 passing unit tests, these show that the prompt → tool call → result → event-stream pipeline works as designed.

Component benchmark of the tool pipeline (measured). The chain, compiler, deployer and wallet MCP services were started locally (Bun, without Docker; Windows 11) on the seeded `DemoVault` workspace, and the sequence *compile* → *snapshot* → *deploy* → *revert snapshot* was executed 10 times through the MCP interface. Every step succeeded in all 10 iterations (compile 10/10, snapshot 10/10, deploy 10/10, revert 10/10). Latencies are in Table 1; the first compilation is a cold start (2.4 s) and later ones hit the compiler cache.

Tool call	n	Median (ms)	Mean (ms)	Min (ms)	Max (ms)
`chain.start_node`	1	3546.6	3546.6	3546.6	3546.6
`compiler.compile`	10	36.5	273.8	33.4	2404.8
`chain.snapshot`	10	4.8	4.8	3.5	7.1
`deployer.deploy_local`	10	16.5	21.3	14.4	71.5
`chain.revert`	10	4.0	4.0	3.3	5.4

Table 1: Measured latency of the local tool pipeline (10 iterations).

Agent loop with an LLM provider. The agent loop (`runAgentTurn`) was run against the local tool services for the task "compile and deploy `DemoVault`" through the Groq OpenAI-compatible API, using the full system prompt (about 4.9k tokens) and a shortened variant (about 2.6k tokens: local-chain workflow and repair rules only). The observed behaviour per configuration is given in Table 2.

Model (Groq)	Configuration	Observation
`gpt-oss-120b`	Full prompt, all tools	Request size 8,304 tokens (provider limit 8,000 tokens per minute).
`gpt-oss-120b`	Shortened prompt	Request accepted; first tool call (`start_node`) was sent with the optional argument `fork` set to `null`.
`gpt-oss-20b`	Shortened prompt	Same behaviour: optional argument sent as `null` on the first tool call.
`qwen3.8-27b`	Shortened prompt, all tools	Request size 7,671 tokens (provider limit 7,000 input tokens per minute).
`qwen3.8-27b`	Shortened prompt, chain + compiler + deployer tools only	Two tool calls issued and answered through MCP (`start_node`, `compile`); the following request, carrying the compile output, was 23,375 tokens.

Table 2: Observations from the agent loop with an LLM provider (one attempt per configuration).

9. Responsible AI Analysis

Risk	Present? (Yes/No)	Possible Impact	Mitigation

Bias	Yes (limited)	The LLM may favour particular libraries, patterns or code idioms learned from its training data, and may produce biased natural-language explanations; the code-generation domain limits direct harm to people.	The system prompt prescribes fixed workflows and toolchain (Hardhat, viem/wagmi); the user can inspect and edit every generated file.
Hallucination	Yes	The model can invent non-existent functions, tools or APIs, or write Solidity that compiles yet is logically wrong or insecure (hallucination [11]).	Output is checked against ground truth: compilation, `simulate_local`, EVM traces and re-execution of the failing transaction in a snapshot; schema-validated tool arguments; the prompt forbids inventing KeeperHub tools or dashboard URLs.
Privacy	Yes	Prompts, workspace source and chat history are sent to third-party inference providers; help requests to the peer mesh carry revert traces and contract source.	Prompt forbids broadcasting private keys or secrets; per-workspace isolation; workspaces are user-owned (foreign IDs return 404); the active provider is shown in the UI. Users should not place secrets in prompts, since prompt content leaves the machine.
Copyright / IP	Yes	Generated code may reproduce licensed code from the training data, and AI-assisted development raises attribution questions.	The README documents which AI tools assisted development ("AI Tool Attribution"); generated contracts should be reviewed and licensed deliberately by the user.
Misuse / Security	Yes	An agent that executes shell commands, signs transactions with operator-configured keys and runs generated frontend code could be abused or manipulated by prompt injection.	Docker runner per workspace; runner ports bound to loopback; JSON-RPC method allowlist (no `hardhat_*`/`debug_*`) enforced in three places; exact-origin `postMessage` bridge (EIP-1193 provider [12]) so private keys never enter the preview iframe; deployment tools target local or testnet chains only; peer patches are verified locally before use.

Human Oversight. A human must review generated Solidity before any deployment outside a throw-away local or testnet chain, because passing compilation and the repair loop does not prove the contract is secure or matches intent; contracts that hold real value require an independent audit. The user should also approve operator-level actions (funding and configuring the deployer key, enabling KeeperHub automation) and check the Inspector events (`patch_proposed`, `patch_verified`, `repair_failed`) to confirm what a repair changed. A patch received from a peer node must be verified (`verify_peer_patch`) before it is applied, as the system prompt requires.

10. Limitations

1. **Model dependence and bounded repair.** Output quality depends on the chosen LLM and is non-deterministic. The repair loop only reacts to reverts of `deploy_local`, `call_contract` and `send_tx_local`, stops after three attempts, and cannot detect logic bugs that do not revert.
2. **Unfinished integrations.** The project's own checklist (`todo.md`) records that the end-to-end KeeperHub demo, the cross-process peer-mesh (Gensyn AXL) proof, and a dedicated mesh panel in the UI are not completed; in the shipped loop a memory miss falls back to LLM reasoning.
3. **Heavy infrastructure and security caveats.** Running the PoC needs Docker, PostgreSQL and inference credentials. The preview bridge still posts messages with target origin `'*'` until the planned preview-subdomain gateway lands (acknowledged in `docs/ARCHITECTURE.md`), and the runner services must never be exposed beyond loopback.
4. **Token footprint and limited test coverage.** A single agent request is about 8,300 tokens, and the compile result raised the context to 23,375 tokens in the measured run (Section 8.3). The repository has unit tests for helper logic but no benchmark of generation success or hallucination rate, and no automated integration test of the full repair loop.

11. Future Scope

1. Complete and demonstrate the peer-mesh knowledge sharing across two independent nodes, add a mesh panel to the UI, and add reputation and signing for mesh patches (listed in the project checklist).
2. Record the full KeeperHub flow (deploy to a testnet, then draft, create and execute an automation workflow) and add the hosted gateway so that previews run on their own origins.
3. Build an evaluation harness (proposed by us, not yet in the repository) with a fixed set of prompts and seeded bugs to measure compile success, repair success rate, number of repair attempts, latency and token cost per model, add integration tests for the repair loop, and shrink tool results (for example, return a compact summary instead of the full compile output) so the agent fits smaller token budgets.

12. Conclusion

This PoC shows that an LLM becomes substantially more useful for smart-contract development when it is treated as a controller of real tools rather than a text generator. Crucible combines a carefully engineered system prompt, schema-validated MCP tools, an inspectable per-workspace runtime and a forced seven-phase repair state machine, so that compiler errors, EVM traces and re-executed transactions—not the model's confidence—decide whether a generated contract is accepted. The agent package's automated tests pass (40 of 40), which supports the correctness of the revert-detection and routing logic; end-to-end quality of generation, the peer mesh and KeeperHub automation still have to be demonstrated and measured. Responsible-AI risks (hallucination, privacy of prompts, code provenance, and agent security) are acknowledged and partly mitigated by design, and human review remains mandatory before any value-bearing deployment.

13. References

[1] *Crucible: Agentic Web3 Development Environment (source repository, `README.md` and `docs/`).* URL: `https://github.com/WhyAsh5114/Crucible`.

[2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations (ICLR)*. 2023.

[3] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. "Toolformer: Language Models Can Teach Themselves to Use Tools". In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. 2023.

[4] *Model Context Protocol Specification*. Model Context Protocol project. URL: `https://modelcontextprotocol.io`.

[5] *Solidity Documentation*. Solidity Team. URL: `https://docs.soliditylang.org`.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. 2017.

[7] *AI SDK Documentation*. Vercel. URL: `https://ai-sdk.dev`.

[8] *Hardhat Documentation*. Nomic Foundation. URL: `https://hardhat.org`.

[9] *0G Compute Network: Router Overview*. 0G Labs. URL: `https://docs.0g.ai/developer-hub/building-on-0g/compute-network/router/overview`.

[10] *EIP-4361: Sign-In with Ethereum*. Ethereum Improvement Proposals. URL: `https://eips.ethereum.org/EIPS/eip-4361`.

[11] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. "Survey of Hallucination in Natural Language Generation". In: *ACM Computing Surveys* 55.12 (2023).

[12] *EIP-1193: Ethereum Provider JavaScript API*. Ethereum Improvement Proposals. URL: `https://eips.ethereum.org/EIPS/eip-1193`.

14. Student Contribution

Student	**Contribution**
Student 1 — Chaitanya Kelkar (Roll No. 15)	Agent and LLM layer (`packages/agent`): system-prompt engineering, tool-calling loop, forced seven-phase repair state machine, 0G/OpenAI-compatible inference routing; unit tests for this package and the matching report sections.
Student 2 — Jay Kerkar (Roll No. 16)	MCP runtime services and runner (`mcp-chain`, `mcp-compiler`, `mcp-deployer`, `mcp-wallet`, `mcp-memory`, `mcp-mesh`, `mcp-terminal`, Docker runtime image); their tests and the matching report sections.
Student 3 — Yash Kolekar (Roll No. 17)	Backend control plane and web UI (`packages/backend` API, authentication, workspace management, preview server; `packages/frontend` SvelteKit workspace and EIP-1193 preview bridge); their tests and the matching report sections.

15. Faculty Assessment — 10 Marks

Assessment Criterion	**Marks**	**Marks Awarded**
Problem identification and relevance	1	
Understanding of selected GenAI concept	1	
System architecture / design	1	
Working implementation	2	
Demonstration and output	2	
Evaluation and analysis	1	
Responsible AI considerations	1	
Viva / individual understanding	1	

Total: 10 Marks Marks awarded (total): _____